

CP020003 — Artificial Intelligence 2026

Khon Kaen University

Week 11— Homework Assignment

In class this week (Week 11) we walked through six computer vision tasks using one consistent library (Ultralytics YOLOv8) plus a pretrained depth model (MiDaS): image classification, object detection, oriented object detection (OBB), instance segmentation, monocular depth estimation, and pose estimation — training each on a tiny toy dataset and scoring it with the metric family that task is normally judged on.

This assignment asks you to take one of those tasks off the toy dataset and onto the real internet: find your own open dataset, train a model, and report proper metrics, exactly as we did in class.

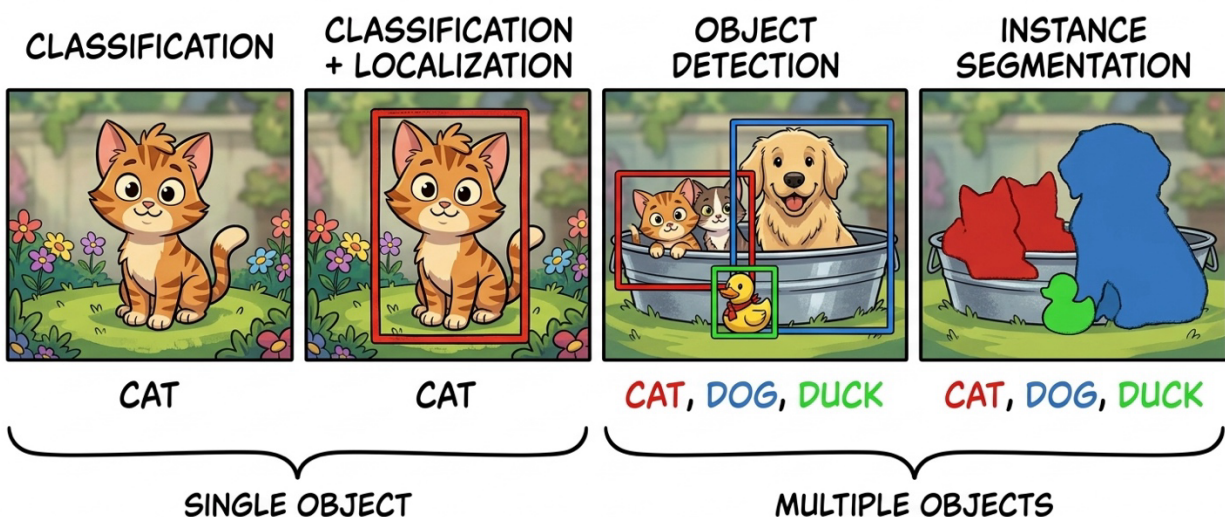
1. Choose Your Task 🎯

Pick exactly ONE of the six computer vision tasks covered in class:

- Image Classification
- Object Detection
- Oriented Object Detection (OBB)
- Instance Segmentation
- Monocular Depth Estimation
- Pose Estimation

State which task you picked and, in 1–2 sentences, why it interests you (a real product, hobby, or research question is a great motivation).

COMPUTER VISION TASKS



2. Find an Open Dataset 🔍

Search the internet for a public dataset that fits your chosen task and is NOT one of the toy datasets used in class (mnist160, coco8, dota8, coco8-seg, coco8-pose). Good places to look:

- Kaggle Datasets
- Roboflow Universe (great for detection/segmentation/pose/OBB — exports directly to YOLO format)
- Hugging Face Datasets
- Google Dataset Search
- Official benchmark pages (e.g. COCO, NYU Depth V2, KITTI, ImageNet subsets)

Report: dataset name and link, number of images, number of classes/categories, and how you split it into train/validation.

3. Build & Evaluate Your Model 🔧

Follow the same workflow as the in-class notebook for your chosen task:

1. Setup — load and inspect the dataset; show a few sample images with their labels.
2. Prepare the data in the label format your task needs (see the "Use Your Own Dataset" guide for that task in the Week 11 notebook).
3. Train a model (a pretrained YOLOv8 nano checkpoint fine-tuned on your data is perfectly fine; any other reasonable architecture is also fine).
4. Run inference and visualize predictions on a handful of validation images.
5. Score the model with the standard metric(s) for your task (see table below) and comment on what the numbers mean in plain language.
6. Conclusion — what worked, what didn't, and one idea you'd try next to improve results.

Task	Example open-source models	Headline metric(s)
Image Classification	ResNet, EfficientNet, YOLOv8-cls, ViT (timm)	Top-1 / Top-5 accuracy
Object Detection	YOLOv8/v11, Faster R-CNN, RT-DETR	Precision, Recall, mAP50, mAP50-95
Oriented Object Detection (OBB)	YOLOv8-obb	mAP50, mAP50-95 (rotated IoU)
Instance Segmentation	YOLOv8-seg, Mask R-CNN, SAM (as a helper)	Box mAP + Mask mAP50/50-95
Monocular Depth Estimation	MiDaS, DPT, Depth Anything	AbsRel, RMSE, delta l
Pose Estimation	YOLOv8-pose, MediaPipe Pose, HRNet	Box mAP + Pose mAP (OKS)

⚠ Grading Emphasis — Read This Carefully

Getting a model to simply run is not enough. Most marks come from: choosing a dataset that genuinely fits your task, reporting the correct metrics (not just accuracy for everything), showing real prediction examples (including at least one failure case), and writing a clear, numbers-backed conclusion.

4. Bonus Points 🌟

Collect your own data — take your own photos or record your own video/images instead of using an internet dataset, label it yourself, and train your model on it — **+2% on your raw grade (out of 100%) for grading.**

This bonus applies on top of Section 3 above — you still need to train and evaluate the model properly.

Submission

Submit your notebook (.ipynb) with all cells run and outputs visible, plus a short written summary (dataset, task, metrics, conclusion) of no more than one page.